

Clase 08 (05/12/2026)

1. URDF

- Archivo en formato XML (etiquetas anidadas) estandarizado para describir la geometría y características de un robot en general. wiki.ros.org/urdf/XML
- Estructura de árbol basado en dos elementos fundamentales: **links** que representan las partes y **joints** que representa cómo están relacionadas esas partes.
- 3 etiquetas principales:

- **robot**: Etiqueta raíz donde se ubica el contenido del URDF. Un solo atributo **name**.

```
<?xml version="1.0"?>
<robot name="[nombre]">
  ...
  <!-- Contenido URDF -->
  ...
</robot>
```

- **link**: Un solo atributo **name**. Se definen 3 geometrías o características:

- Visual: Origen, forma y material (color)

```
<link name="[nombre]_link">
  ...
  <visual>
    <origin xyz="[x] [y] [z]" rpy="[roll] [pitch] [yaw]" />
    <geometry>.. </geometry>
    <material>..</material>
  </visual>
  ...
</link>
```

- Colisión: Origen y geometría

```
<link name="[nombre]_link">
  ...
  <collision>
    <origin xyz="[x] [y] [z]" rpy="[roll] [pitch] [yaw]" />
    <geometry>.. </geometry>
  </collision>
  ...
</link>
```

- Inercial: Origen, masa e inercia

```
<link name="[nombre]_link">
  ...
  <inertial>
    <origin xyz="[x] [y] [z]" rpy="[roll] [pitch] [yaw]" />
    <mass value="{masa}" />
    <inertia .. />
```

```

    </inertial>
    ...
</link>

```

- **joint**: Dos atributos **name** y **type**. Dos elementos obligatorios: **parent** y **child**. 4 tipos de joints principales:

- **fixed**: Fija (**origin**)
- **continuous**: Continua (**origin** + **axis**)
- **revolute**: Revolución (**origin** + **axis** + **limit**)
- **prismatic**: Prismática

```

<joint name="[nombre]_joint" type="[tipo]">
  <parent link="[nombre_padre]"/>
  <child link="[nombre_hijo]"/>
  <origin xyz="[x] [y] [z]" rpy="[roll] [pitch] [yaw]"/>
  <axis xyz="[x] [y] [z]"/>
  <limit .. />
</joint>

```

- Uso de joints fijas y partes NO móviles

1.1. XACRO

- Acrónimo de *XML Macro*. Presenta utilidades para facilitar la codificación de URDF, como división de código, estructuras de control o parámetros. wiki.ros.org/xacro.
- Se añade un paso de compilación a través del ejecutable **xacro**.
- División sugerida: Core con la geometría (**links** y **joints**); Macros; Materiales; Sensores (**links** y **joints**); Gazebo y parámetros de simulación.
- Archivo principal con el tag **robot**, el nombre (es lo que lo diferencia de un archivo URDF) y el atributo **xmlns:xacro**. Generalmente se nombra con extensión **.urdf.xacro**.
- Incluir archivos con **xacro:include**. Los archivos incluidos tienen el tag **robot** pero sin el atributo de nombre.

```

<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="[nombre]">
  <xacro:include filename="[archivo].xacro" />
  ...
</robot>

```

- Propiedades **xacro:property** y operaciones matemáticas **\${..}**
- Argumentos **xacro:args**: Nombre y valor por defecto
- Bloques condicionales **<xacro:if value="[expresion]">..**</xacro:if>**** y **<xacro:unless value="[expresion]">..**</xacro:unless>****
- Macros **xacro:macro**
- Compilación del URDF:

- desde consola: **\$ xacro nombre_archivo.xacro.urdf**
- desde *launch*:

```

setup(
  name=package_name,
  # ...
  data_files=[
    # ...

```

```

        # Install launch files inside 'launch' folder
        (os.path.join('share', package_name, 'launch'),
         glob(os.path.join('launch', '*.launch.py'))),
        # Install URDF and XACRO files inside de 'urdf' folder
        (os.path.join('share', package_name, 'urdf'),
         glob(os.path.join('urdf', '*[.urdf][.xacro]'))),
        # Install STL files inside de 'meshes' folder
        (os.path.join('share', package_name, 'meshes'),
         glob(os.path.join('meshes', '*.stl'))),
    ],
    ...
)

```

- Adaptación estructura de paquete. Configuración `setup.py`:

```

# Ubicación del paquete y del archivo URDF
urdf_path = PathJoinSubstitution(
    [FindPackageShare("[nombre_paquete]"), "urdf", "[nombre_archivo].urdf.xacro"]
)

# Procesar archivo URDF
urdf = Command(['xacro ', urdf_path])

```

1.2. URDF y tf2

- Similitud en las estructuras. Conversión entre juntas de un tipo a su respectiva transformación.
- Uso del paquete `robot_state_publisher` para transformaciones estáticas y dinámicas, y para publicar el topic `robot_description`

- desde consola: `$ ros2 run robot_state_publisher robot_state_publisher --ros-args -p robot_description:='<robot_description>'`
- desde *launch*:

```

Node(
    package = 'robot_state_publisher',
    executable = 'robot_state_publisher',
    parameters=[{
        'robot_description': '<robot_description>',
    }]
)

```

- Particularidad de las juntas no fijas: Datos necesarios para publicar la transformación. Uso del paquete `joint_state_publisher_gui` para simular posiciones de las juntas.

- desde consola: `$ ros2 run joint_state_publisher_gui joint_state_publisher_gui`
- desde *launch*:

```

Node(
    package = 'joint_state_publisher_gui',
    executable = 'joint_state_publisher_gui',
    output = 'screen',
)

```